

P30

Reflexion: agentes de lenguaje con refuerzo verbal

Reflexion: Language Agents with Verbal Reinforcement Learning

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, Shunyu Yao · 2023 · arXiv:2303.11366 ·
NeurIPS 2023

Ficha pedagógica del Artificial Intelligence Evolution Program. No es el paper original: es una guía de lectura en español
que enlaza a la fuente primaria. Consultado 2026-08-16.

P30 — Reflexion

Ruta de agentes · El agente aprende entre intentos sin tocar un solo peso: el refuerzo ocurre en el contexto, en lenguaje natural.

Nivel: L2 · Motor: `reflexion` · Notebook: `P30_reflexion.ipynb` · Anexo: cálculo y gradientes

1. Identificación

Campo	Valor
Título original	<i>Reflexion: Language Agents with Verbal Reinforcement Learning</i>
Autoría	Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, Shunyu Yao
Año	2023
Venue	arXiv:2303.11366 · NeurIPS 2023
Fuente primaria	arXiv:2303.11366
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Un bucle ReAct que falla vuelve a empezar de cero. No conserva nada del intento anterior, así que **repite el mismo error** con la misma confianza.

La alternativa clásica sería ajustar los pesos con refuerzo, pero eso es caro, lento y exige acceso al modelo. Para un agente construido sobre una API, ni siquiera es posible.

3. Propuesta

- Cerrar el bucle **en el contexto**. Tras cada intento fallido:
- una señal de evaluación indica que falló (test que no pasa, error del entorno, heurística);
 - el modelo genera una **reflexión verbal** sobre qué salió mal;
 - esa reflexión se guarda en una memoria episódica;
 - el siguiente intento se condiciona con ella.

Es refuerzo en el sentido de que la política mejora con la experiencia, pero la «actualización» es texto añadido al contexto. Cero gradientes.

4. Intuición sin fórmulas

Un agente sin memoria de sus fallos es alguien que repite el mismo error con entusiasmo. Reflexion añade lo mínimo para romper el bucle: escribir qué salió mal y leerlo antes de reintentar.

Dónde deja de funcionar la analogía: una persona reflexiona aunque nadie le diga que falló. Aquí la reflexión necesita una señal externa; sin ella, degenera en autoafirmación.

5. Matemática mínima

No hay ecuación. Lo que hay es un bucle con estado:

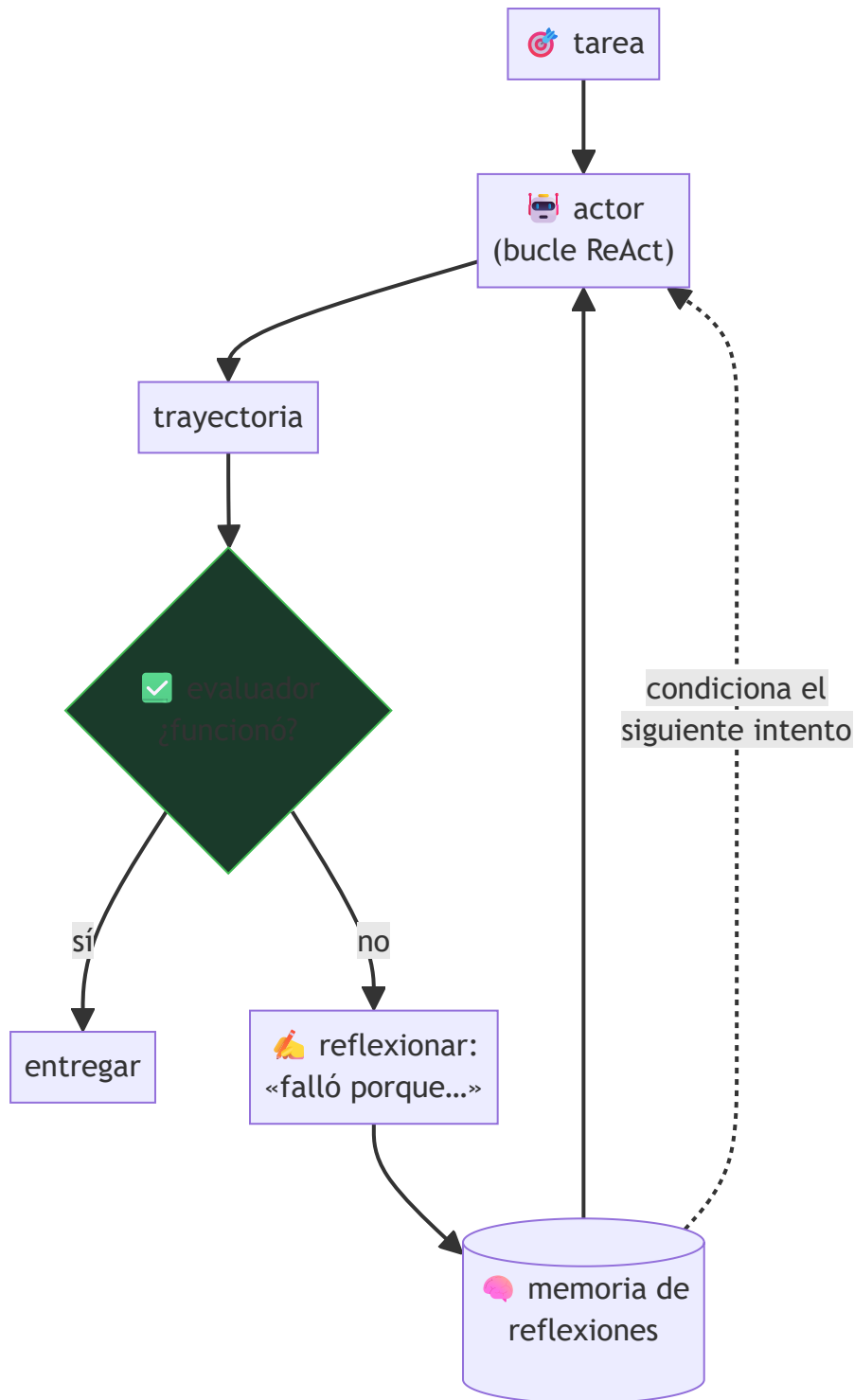
```
memoria = []
para intento en 1..N:
    trayectoria = política(tarea, memoria)      ← el contexto incluye las reflexiones
    resultado   = evaluar(trayectoria)          ← señal EXTERNA, verificable
    si resultado == éxito: terminar
    memoria.append( reflexionar(trayectoria, resultado) )
```

La comparación con el refuerzo clásico es directa: donde RL actualiza θ , aquí se actualiza el **contexto**. Es más barato e inmediato, y también más frágil y efímero.

[!TIP] **Puente matemático.** Esta sección da por sabido lo siguiente. Si algo no te suena, léelo primero: está explicado una sola vez, en un solo sitio, y sirve para todas las fichas.

Dónde	Qué necesitas de ahí
A02 §2 · Entropía	cuánta información aporta un intento fallido sobre el siguiente

6. Arquitectura o flujo



7. Qué observar en el paper original

- Los **tres tipos de señal de evaluación** que consideran, según la tarea: exacta (tests), heurística y generada por el propio modelo. La calidad del método baja con la calidad de la señal.
- Las **curvas de éxito acumulado por número de intentos**: es la forma correcta de reportar un método iterativo, no una sola cifra.
- Los ejemplos de **reflexiones inútiles** («ser más cuidadoso»), que es el modo de fallo característico.

- La comparación con simplemente **reintentar sin reflexión**, que es la línea base honesta: parte de la mejora viene solo de tener más intentos.

8. Evidencia y resultados

Evaluación en tareas de toma de decisiones, razonamiento y generación de código, comparando con ReAct y con reintentos sin reflexión.

Las tasas de éxito por tarea y por número de intentos están en el artículo. Verificarlas allí, y comparar siempre contra «reintentar N veces sin reflexión»: sin esa columna, la mejora atribuida a la reflexión está inflada.

La miniatura de este eje muestra el mecanismo: sin memoria, el agente repite el primer fallo y no termina en cuatro intentos; con memoria verbal, elimina un error por intento y converge. Con cero pesos actualizados.

9. Impacto

- Instaló la idea de **memoria entre intentos** como componente estándar de un agente.
- Es uno de los antecedentes directos del bucle de autocrítica que hoy llevan casi todos los sistemas de generación de código.
- Marcó la distinción entre aprendizaje **en parámetros** y aprendizaje **en contexto** como una decisión de diseño, no como una limitación.

10. Limitaciones

1. **Depende de una señal de fallo fiable.** Sin verificador no hay sobre qué reflexionar.
2. **La reflexión puede ser incorrecta o genérica**, y entonces empeora el siguiente intento.
3. **La memoria ocupa contexto** y crece con los intentos: no escala a cientos.
4. **Efímero**: lo aprendido se pierde al terminar la sesión, salvo que se persista aparte.
5. **Coste multiplicado** por el número de intentos.
6. **Riesgo de bucle**: si la reflexión se repite, el agente puede quedar atrapado.

11. Errores comunes

Error	Corrección
«El modelo aprende»	Aprende en el contexto . Cierra la sesión y no queda nada.
«Basta con pedirle que revise su respuesta»	Sin señal externa, la autocrítica tiende a la autoafirmación o al cambio aleatorio.
«La mejora es de la reflexión»	Parte viene de tener más intentos. Hay que comparar contra reintentar sin reflexionar.
«Es aprendizaje por refuerzo»	Comparte la estructura (política, señal, mejora) pero no actualiza parámetros. La analogía es útil y también engañosa.
«Cuantos más intentos, mejor»	Con memoria creciente y sin criterio de parada, el coste crece y la calidad se estanca.

12. Relación con trabajos anteriores

- **P13 ReAct (2022)** — el bucle que se envuelve.
- **P28 Chain-of-Thought (2022)** — el razonamiento explícito.
- **P26 DQN (2015)** — el marco de refuerzo con el que se hace la analogía.

13. Relación con trabajos posteriores

- **Madaan et al. (2023), Self-Refine** — refinamiento iterativo sin señal externa; el contraste útil. [arXiv:2303.17651](#)
- **P31 Generative Agents (2023)** — memoria que persiste entre tareas, no solo entre intentos.
- **P16 Sistemas agentic** — la síntesis operativa.

14. Notebook asociado

P30_reflexion.ipynb

Qué implementa: el bucle de reintentos con y sin memoria verbal, el conteo de pesos actualizados (cero) y el análisis de cuánto contexto ocupa la memoria al crecer los intentos.

Qué NO implementa: las reflexiones son una lista escrita a mano. En el paper las genera el modelo, y pueden ser inútiles — que es justo el modo de fallo interesante.

```
ai-evolution paper-lab P30 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe los cuatro pasos del bucle.
Explicar	Explica por qué hace falta una señal externa.
Aplicar	Ejecuta el notebook y compara ambos bucles.
Analizar	Calcula cuántos intentos caben en un contexto de 8 000 tokens con reflexiones de 80.
Evaluar	Un equipo reporta +15 % con Reflexion. ¿Qué línea base exiges?
Crear	Diseña un detector de reflexión inútil («ser más cuidadoso») y di qué haría el agente al detectarla.

16. Autoevaluación

1. ¿Qué se actualiza exactamente en cada iteración?
2. ¿Por qué se le llama refuerzo si no hay gradientes?
3. ¿Qué pasa sin señal de evaluación externa?
4. ¿Cuál es la línea base correcta para medir la mejora?
5. ¿Qué límite impone el tamaño del contexto?
6. ¿Cómo se evita que el bucle no termine?

7. ¿Qué diferencia hay entre esta memoria y la de Generative Agents?

17. Respuestas esperadas

1. El **contexto**: se añade una reflexión a la memoria que condiciona el siguiente intento. Los parámetros no cambian.
2. Porque comparte la estructura del refuerzo —política, señal de resultado, mejora iterativa— aunque el mecanismo de actualización sea textual.
3. La reflexión no tiene información nueva que incorporar: el modelo tiende a declararse satisfecho o a cambiar cosas al azar.
4. Reintentar el mismo número de veces **sin** reflexión. Parte de la mejora viene solo de tener más oportunidades.
5. La memoria crece con los intentos y compite por el contexto con la tarea. A partir de cierto punto hay que resumir o priorizar.
6. Con límite de intentos, detección de reflexiones repetidas y escalamiento a un humano.
7. Reflexion recuerda entre **intentos de la misma tarea**; Generative Agents mantiene memoria **entre tareas y a lo largo del tiempo**, con recuperación puntuada.

18. Fuentes primarias

- Shinn, N. et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. **NeurIPS 2023**. arXiv:2303.11366 · consultado 2026-08-16.
- Madaan, A. et al. (2023). *Self-Refine: Iterative Refinement with Self-Feedback*. arXiv:2303.17651 · consultado 2026-08-16.



Evaluación — P30 · Reflexion: agentes de lenguaje con refuerzo verbal

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Reflexion: Language Agents with Verbal Reinforcement Learning* (2023, arXiv:2303.11366 · NeurIPS 2023) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P30_reflexion.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).